Pallavi Das
Kasey Liu

CSC 482 Final Project Report
Project 4: Detect Names from legislative speech self-introductions

Problem, Background, and Context

Legislative hearings are an important part of the democratic process, but their transcripts are often difficult for the public to follow. Many are generated using Automatic Speech Recognition (ASR), which introduces errors like misspelled names and inconsistent formatting. A key barrier to accessibility is identifying who is speaking at any given moment, especially when working from transcripts alone, without supporting audio or video.

Named Entity Recognition (NER) is a Natural Language Processing technique that identifies and classifies names and other entities in text, and could help make these transcripts more structured and readable. However, most NER models are not specifically trained on legislative transcript data, and speaker identification tools are typically built for other use cases, meaning neither performs well in this domain.

This project addresses both gaps by building a specialized NER model that detects and classifies human names in state legislative hearing transcripts while correctly attributing each utterance to its current speaker. To handle self-introductions, we trained a BERT-based binary classifier to detect when a speaker is introducing themselves.

Data Sources

We are using the Digital Democracy 2015-2018 dataset from Hugging Face, a large corpus of comprehensive proceedings of state legislative hearings spanning the California, Texas, New York, and Florida.

Kind of data:

There are hearing transcripts (the text of what was spoken in legislative hearings) and metadata fields (speaker metadata, legislative bill metadata, and relational data connecting utterances to speakers, speakers to committees, hearings to bills and videos, and sessions to states). We will analyze the utterances in the speeches.csv file.

Size of data:

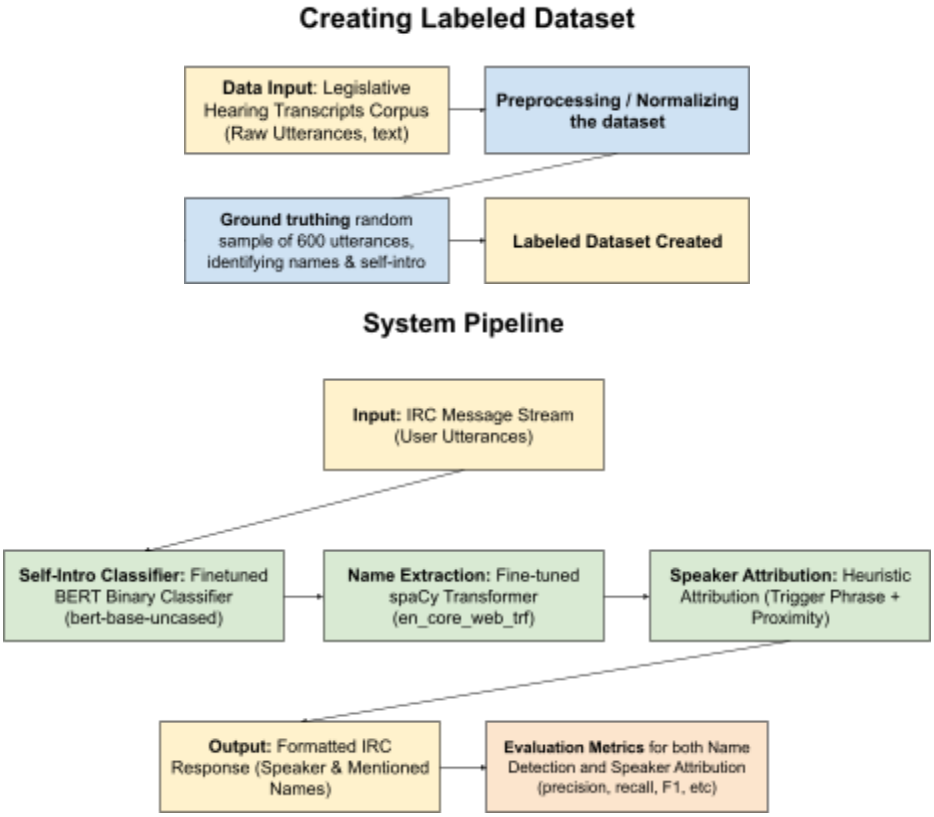
The corpus contains over 1.6 million utterances/speeches from about 116,000 individually identified speakers across four state legislatures. The data is over 1.66GB uncompressed and 344.53 MB compressed. For testing, we loaded and parsed the California 2017-2018 session subset from speeches.csv, which contains 392,565 utterances.

Format of data:
The corpus is a series of CSV files that closely resemble the tables of a relational database: utterances, people, committees, bills, hearings, discussions, and videos.

Tools and Services

- **BERT** (bert-base-uncased): the base model fine-tuned for self-introduction classification
- **spaCy** (en_core_web_trf): the base NER model fine-tuned for person name detection in legislative transcripts
- **Hugging Face Transformers**: used to load and fine-tune the BERT model and tokenizer via AutoModelForSequenceClassification and AutoTokenizer
- **PyTorch**: powers the training loop, data loading, and model inference for BERT
- **scikit-learn**: used for train/test splitting with stratification
- **pandas**: used for loading and processing the labeled CSV dataset
- **numpy**: used for random seed setting during training
- **Hugging Face Datasets**: Digital Democracy 2015–2018 corpus sourced from Hugging Face
- **IRC (irc.libera.chat)**: the chatbot runs on the Libera. Chat IRC server using the IRC Python library

Design of our system



Data Access and Preprocessing
We began by programmatically accessing the Digital Democracy 2015–2018 corpus and loading all speeches.csv files from the available state-session folders. We verified the column structure and computed the total row count to confirm the dataset size and relational structure, which links speeches to speakers via person IDs. We then loaded the relevant columns, including speech ID, person ID, hearing ID, hearing date, session year, first name, last name, and transcript text, into a structured DataFrame, skipping any malformed rows. We added session and state identifiers, trimmed whitespace, cast all fields to strings, and created a speaker_name field by combining first and last names. We also standardized transcripts, normalized honorifics and name variants (e.g., making "Senator" and "Dr." consistent), tokenized text, and performed structured segmentation. We then generated labeled data distinguishing the current speaker from other individuals mentioned in the transcript to support both NER and speaker attribution model training.

Labeling the Dataset
To support model training, we created a labeled dataset of 599 utterances drawn from the California 2017–2018 session of the Digital Democracy Corpus. Each utterance was annotated with four fields: the true speaker name, all names mentioned in the utterance, whether the utterance was a self-introduction, and the name used in the self-introduction if applicable. True speaker names were generated by concatenating first and last names from the dataset, while the remaining fields were labeled manually. The resulting dataset contained 116 self-introductions (19%), 483 non-introductions (81%), 332 utterances with at least one name (55%), and 267 utterances with no names (45%). Here is our ground truth dataset labeled manually:
NER Legislative Labeled Dataset

Self-introduction Classifier
Our initial approach to identifying self-introductions used handwritten regular expression patterns, such as "I am" and "My name is." While this worked in straightforward cases, it produced many false positives and missed less obvious patterns. To improve accuracy, we trained a BERT-based binary classifier using the Hugging Face bert-base-uncased model. The classifier took an utterance as input and predicted whether it contained a self-introduction. Our labeled dataset was split into training and validation sets, and the resulting sets were fed into a PyTorch training loop that fine-tuned the model for 3 epochs using the AdamW optimizer. After training, the model and tokenizer were saved for use in real-time predictions.

spaCy NER Model
To improve name detection in legislative transcripts, we fine-tuned a spaCy en_core_web_trf model, a RoBERTa-based transformer, on our manually labeled dataset. We converted the all_names annotations in each utterance into spaCy's training format with character-offset

entities, generating multiple name format possibilities (first name, last name, full name) to help the model learn partial name recognition. The labeled data was then split 80/20 into 478 training and 120 evaluation examples.

During the fine-tuning, we froze non-essential pipeline components like the tagger, parser, attribute_ruler, and lemmatizer, so that only the NER head and transformer were updated. This is because our training dataset only contained labels for PERSON entities. Updating the other components without corresponding labels introduced noise into their pre-trained weights and lowered the F1 score, potentially causing further catastrophic forgetting of the model's foundational English grammar. Freezing the nonessential pipes allows the training to focus entirely on the NER head and transformer embeddings so that the model can be specialized for legislative name detection. We then fine-tuned the pre-trained weights for 20 epochs with compounding batch sizes of 4 to 16 and 0.2 dropout. After training, the model was saved to be used by the bot for real-time extraction similar to the BERT model.

IRC Chatbot Integration

We integrated our system into an IRC chatbot that allowed users to input a transcript utterance and receive predictions in real time. The chatbot ran the spaCy NER model to extract all person names from the input, ran the BERT classifier to determine whether the utterance was a self-introduction, and returned the detected names along with a predicted speaker name. A heuristic speaker attribution module uses trigger phrases to identify which detected name belongs to the speaker versus names that are merely mentioned.

<u>Evaluation and Testing</u>

We evaluated our system both manually and using metrics such as accuracy, precision, recall, and F1 score.

These were the results from our baseline experiment (no training).

Name Extraction w/ spaCy NER (PERSON Entities)

Precision	Recall	F1	TP	FP	FN
0.8045	0.7226	0.7614	568	138	218

Self-Introduction Detection w/ Regex Patterns

Accuracy	Precision	Recall	F1
0.7129	0.3444	0.5345	0.4189

TN = 365	FP = 118
FN = 54	TP = 62

These were the results from our experiment using our trained BERT binary classifier and our trained spaCy NER model:

Self-Introduction Detection w/ BERT Binary Classifier

Accuracy	Precision	Recall	F1
0.9900	0.9583	0.9914	0.9746

TN = 478	FP = 5
FN = 1	TP = 115

Name Extraction w/ Fine-tuned spaCy NER (en_core_web_trf)

Precision	Recall	F1
0.9460	0.9520	0.9490

This represents a +24.8% F1 improvement over the untrained baseline (0.7614 -> 0.9490).

Here are some example inputs that we fed into our system, which are from the California 2015-2016 session:

Example 1 - Self-introduction with multiple names (multi-part, 881 characters):

- Input: *classify "Yes, Mr. Chairman and Madam Vice Chair and members the Committee. I'm Tony Gonzalez, not representing any organization today, but representing myself as Geoff's best friend, his best man in his wedding, Geoff was best man in my wedding, and Geoff is the godfather of my daughter. We met thirty three years ago at Claremont McKenna College. What I can tell, right across the street from Pitzer College, we might have even cross paths at that time, who knows where, Collins? And, what I would briefly say, is that I cannot emphasize enough Geoff's passion for public service going back to CSpan. Which, I consider to be a form of public service, to now, I think for at least ten years now involved with the community colleges. First at the PCC and now with the Board of Governors. He's very excited to continue his service, and I know he'll do a fantastic job if confirmed. thank you"*

- Output: *"This is a self-introduction (confidence: 99.5%)! I believe the speaker is Tony Gonzalez. Other names mentioned: Geoff, Collins."*

Example 2 - Non-self-introduction with detected names:

- Input: *classify "Thank you and to be fair this is a question, a similar question that I posed to Mr. Ramos too when I was sitting right over there where Ms. ___ is sitting. So, with that I'm going to open it up to my colleagues, Mr. Cannella."*
- Output: *"I do not think this is a self-introduction (confidence: 99.7%). However, I did detect the following names: Cannella, Ramos."*

Example 3- Short self-introduction:

- Input: *"My name is Peggy"*
- Output: *"This is a self-introduction (confidence: 99.3%)! I believe the speaker is Peggy. Other names mentioned: none."*

Example 4 - Long self-introduction (2,426 characters):

- Input: *classify " Mr. President, members. Carl London here on behalf of the California Disability Services Association. We are a organization state-wide of a hundred community-based non-profits that serve the population served by this department on a daily basis. We're here today, backing up the letter of support we filed in support of Mr. Doyle. We think he's a fine guy for this job. I do want to make a couple comments though, related to some of the issues that we would ask you to keep a close watch on as this moves forward. And it also, as the next appointment comes up, which is for the director's position. There's a few things going on right now, that are quite relevant to what's going to happen with this population and we absolutely look forward working to Mr. Doyle. They're looking forward to working with Mr. Doyle on these issues. The federal government's come down recently with some massive changes proposed to the CMS guidelines that guide the funding relationship we have with the federal government for this population…"*
- Output: "*This is a self-introduction (confidence: 99.7%)! I believe the speaker is Carl London. Other names mentioned: Doyle."*

<u>Shortcomings and Failures</u>

Our models are trained only on California sessions from 2017-2018; they may overfit to this subset. IRC Protocol also has a character limit, meaning long utterances can't be tested unless concatenated. Additionally, a shortcoming of using our trained models is latency. We saw a significant increase in response time while testing with our custom-trained BERT classifier and spaCy NER model. This is a tradeoff for increased accuracy, precision, recall, and F1.

<u>Conclusion and Future Work</u>

Our project demonstrates that a specialized NER and speaker-attribution system can be built for legislative transcripts with a relatively small amount of labeled data. By fine-tuning both a spaCy NER model and a BERT binary classifier on 599 manually labeled utterances, we achieved greater accuracy than our initial approach, which used untrained models and regex to detect person names and identify self-introductions in transcript text.

The most significant limitation of our current system is the size of our training data. With over 1.6 million speeches across multiple states in the Digital Democracy 2015–2018 Corpus, there is substantial room to expand our dataset and expose our models to a wider range of speakers, states, and linguistic patterns. We expect that training on more data would meaningfully improve accuracy and generalization across sessions and states. We also hope to fine-tune our speaker detection logic further, as it relies heavily on regular expressions. Lastly, we hope to continue optimizing our pipeline for speed, as our trained models introduce increased latency.

More broadly, we hope this work serves as a useful building block for the Digital Democracy project and similar efforts to make legislative proceedings more transparent and accessible to the public.